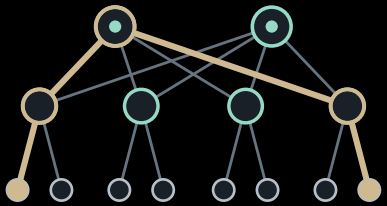
PURDUE UNIVERSITY

STyGIANet

CS 422 / COMPUTER NETWORKS

Topologies



Build · Route · Reconfigure

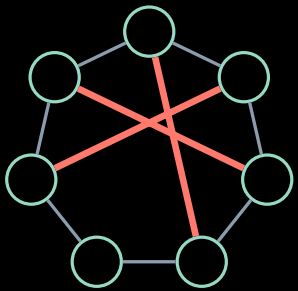
Vamsi Addanki · vaddank@purdue.edu · Fall 2026

What makes a good network topology?

STyGIANet

How should we connect N machines?

- hardware cost and port count
- latency and bandwidth
- resilience to failures
- routing and traffic patterns



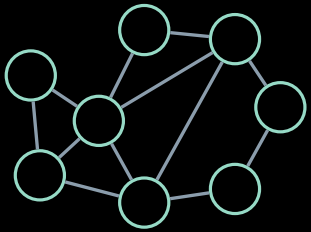
every link is a design choice

Unstructured and structured networks

STyGIANet

Unstructured

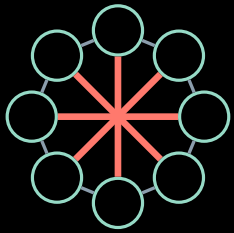
Peers join using local rules.



Easy to join, but node degrees vary.

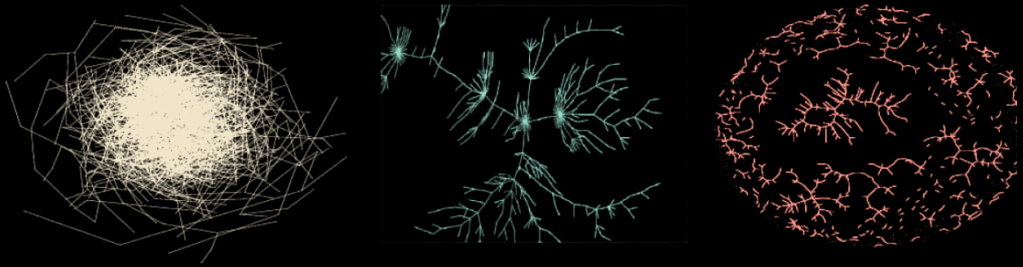
Structured

Fixed rules determine links.



Known links make routing simple.

Gnutella depends on its hubs



Gnutella 2001
Original peer network.

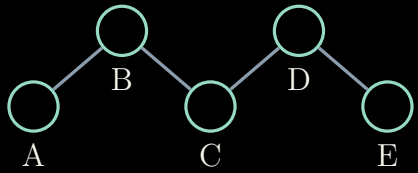
30% random loss
A giant component remains.

4% hub loss
The network fragments.

The graph lens

$G = (V, E)$: machines are vertices and links are edges.

- $\deg(v)$: links incident to v
- $d(u, v)$: shortest path length
- $D(G) = \max_{u,v} d(u, v)$: diameter
- $\Gamma(U)$: neighbors just outside U

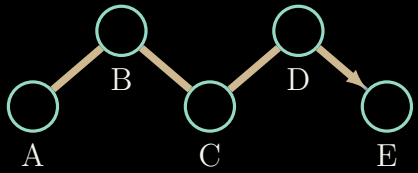


Follow links to form a path.

The graph lens

$G = (V, E)$: machines are vertices and links are edges.

- $\deg(v)$: links incident to v
- $d(u, v)$: shortest path length
- $D(G) = \max_{u,v} d(u, v)$: diameter
- $\Gamma(U)$: neighbors just outside U



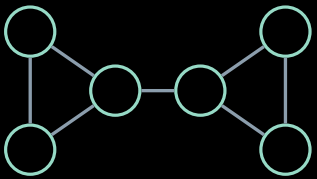
$d(A, E) = 4$; this is a diameter.

Expansion reveals a narrow boundary

For $1 \leq |U| \leq |V|/2$,

$$\alpha(U) = \frac{|\Gamma(U)|}{|U|}, \ \alpha(G) = \min_{U} \alpha(U).$$

Small expansion means that many nodes depend on a small boundary.

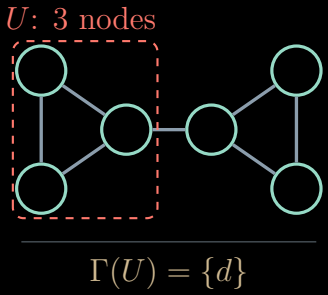


Expansion reveals a narrow boundary

For $1 \leq |U| \leq |V|/2$,

$$\alpha(U) = \frac{|\Gamma(U)|}{|U|}, \ \alpha(G) = \min_{U} \alpha(U).$$

Small expansion means that many nodes depend on a small boundary.



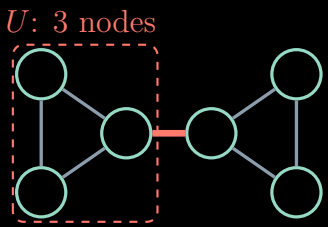
Expansion reveals a narrow boundary

For $1 \le |U| \le |V|/2$,

$$\alpha(U) = \frac{|\Gamma(U)|}{|U|}, \ \alpha(G) = \min_U \alpha(U).$$

Small expansion means that many nodes depend on a small boundary.

Drawback: One boundary failure can separate a large region.



Question: where would one extra link go?

A seven-node path has diameter 6. You may add one cable.



Where would you place it to reduce the worst-case hop count?

The basic tradeoff

Topology	Max degree	Diameter	Narrow cut?
Complete graph	$N-1$	1	no
Line	2	$N-1$	yes
Balanced binary tree	3	$\Theta(\log N)$	yes
Square mesh	4	$\Theta(\sqrt{N})$	moderate
Hypercube	$\log_2 N$	$\log_2 N$	no

Degree, diameter, and cut capacity pull in different directions.

Formal definitions: basic graphs

Complete graph K_N

$V = [N], \qquad E = \{\{i, j\} : 1 \le i < j \le N\}.$

Path P_N

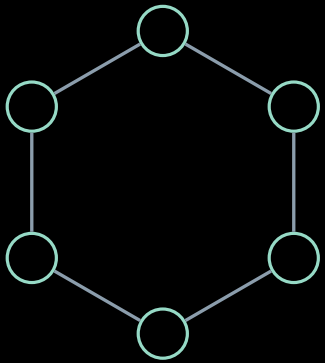
$V = [N], \qquad E = \{\{i, i+1\} : 1 \le i < N\}.$

Full binary tree T_h

$V = \bigcup_{k=0}^{h} \{0,1\}^k$, including the empty string ϵ.

$E = \{\{x, xb\} : |x| < h,\ b \in \{0,1\}\}.$

Complete graph: connect every pair



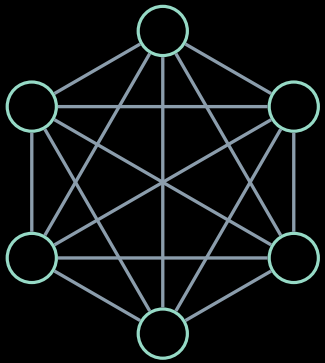
Begin with a ring.

Benefit: Diameter 1 and rich path diversity.

Drawback: $N(N-1)/2$ links and $N-1$ ports per node.

Useful for: Small groups or logical full connectivity through switches.

Complete graph: connect every pair



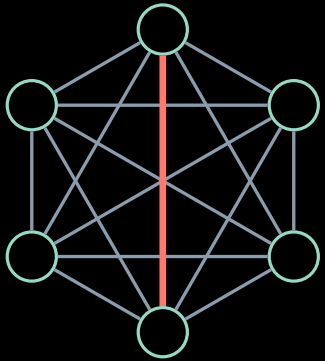
Add all missing links.

Benefit: Diameter 1 and rich path diversity.

Drawback: $N(N-1)/2$ links and $N-1$ ports per node.

Useful for: Small groups or logical full connectivity through switches.

Complete graph: connect every pair



Every destination is now one hop away.

Benefit: Diameter 1 and rich path diversity.

Drawback: $N(N-1)/2$ links and $N-1$ ports per node.

Useful for: Small groups or logical full connectivity through switches.

Line: a minimal connected network



Benefit: Maximum degree 2; simple routing.
Useful for: Pipelines dominated by local traffic.

Drawback: Diameter $N - 1$; a middle link carries many routes and its failure partitions the line.

Line: a minimal connected network



Benefit: Maximum degree 2; simple routing.
Useful for: Pipelines dominated by local traffic.

Drawback: Diameter $N - 1$; a middle link carries many routes and its failure partitions the line.

Line: a minimal connected network



Benefit: Maximum degree 2; simple routing.
Useful for: Pipelines dominated by local traffic.

Drawback: Diameter $N - 1$; a middle link carries many routes and its failure partitions the line.

Binary tree: branching shortens paths

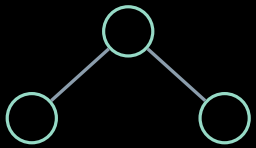


Benefit: Balanced tree: $O(\log N)$ diameter with degree at most 3.

Drawback: Upper edges are bottlenecks; each pair has only one path.

Useful for: Aggregation and control hierarchies.

Binary tree: branching shortens paths

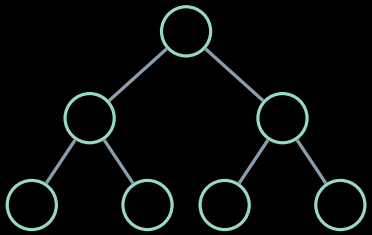


Benefit: Balanced tree: $O(\log N)$ diameter with degree at most 3.

Drawback: Upper edges are bottlenecks; each pair has only one path.

Useful for: Aggregation and control hierarchies.

Binary tree: branching shortens paths

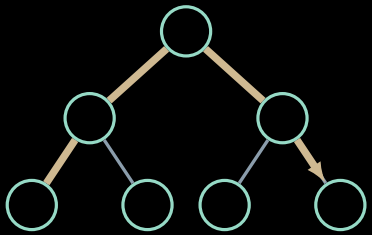


Benefit: Balanced tree: $O(\log N)$ diameter with degree at most 3.

Drawback: Upper edges are bottlenecks; each pair has only one path.

Useful for: Aggregation and control hierarchies.

Binary tree: branching shortens paths

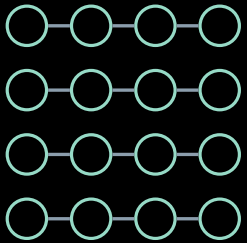


Benefit: Balanced tree: $O(\log N)$ diameter with degree at most 3.

Drawback: Upper edges are bottlenecks; each pair has only one path.

Useful for: Aggregation and control hierarchies.

Mesh: add a second dimension

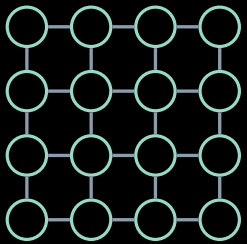


Benefit: Degree at most 4; coordinate routing; local wiring.

Drawback: For $N = s^2$ nodes, diameter $2(s-1) = \Theta(\sqrt{N})$.

Useful for: Chips and arrays with neighbor exchanges.

Mesh: add a second dimension

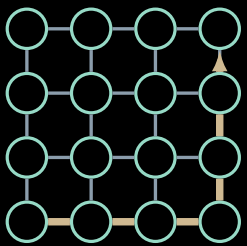


Benefit: Degree at most 4; coordinate routing; local wiring.

Drawback: For $N = s^2$ nodes, diameter $2(s-1) = \Theta(\sqrt{N})$.

Useful for: Chips and arrays with neighbor exchanges.

Mesh: add a second dimension

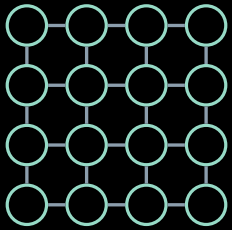


Benefit: Degree at most 4; coordinate routing; local wiring.

Drawback: For $N = s^2$ nodes, diameter $2(s-1) = \Theta(\sqrt{N})$.

Useful for: Chips and arrays with neighbor exchanges.

Torus: close the mesh boundaries



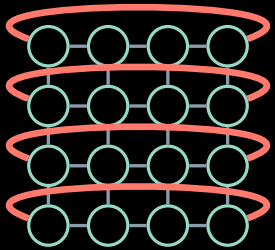
Opposite sides connect.

Benefit: Uniform degree 4; shorter routes.

Drawback: Wraparound cables can be long.

Useful for: Regular HPC fabrics.

Torus: close the mesh boundaries



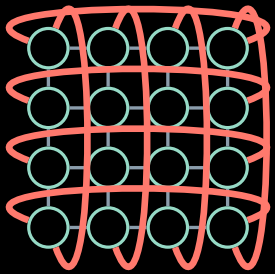
Opposite sides connect.

Benefit: Uniform degree 4; shorter routes.

Drawback: Wraparound cables can be long.

Useful for: Regular HPC fabrics.

Torus: close the mesh boundaries



Opposite sides connect.

Benefit: Uniform degree 4; shorter routes.

Drawback: Wraparound cables can be long.

Useful for: Regular HPC fabrics.

Formal definitions: mesh and torus

Both use coordinates (i, j) with m rows and n columns.

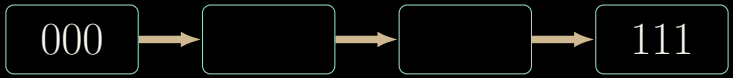
Mesh $M_{m,n}$
$V = [m] \times [n]$; connect u and v exactly when $\|u - v\|_1 = 1$.

Torus $T_{m,n}$**,** $m, n \geq 3$
$V = \mathbb{Z}_m \times \mathbb{Z}_n$. Neighbors of (i, j) are
$(i \pm 1 \bmod m, j)$ and $(i, j \pm 1 \bmod n)$.

The torus adds wraparound edges to the mesh.

Question: can labels replace a route map?

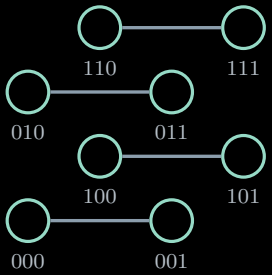
Give every node a three-bit label. Start at 000 and target 111.



one bit changes at each hop

Which pairs of labels must have a link?

Hypercube: add one dimension at a time



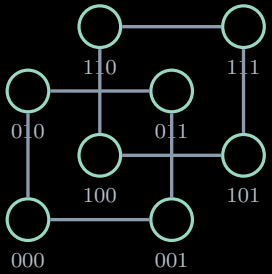
Definition

$V = \{0,1\}^d$. Join x and y iff their Hamming distance is one.

Benefit: Degree and diameter both equal $d = \log_2 N$.

Drawback: Port count grows with $\log N$.

Hypercube: add one dimension at a time



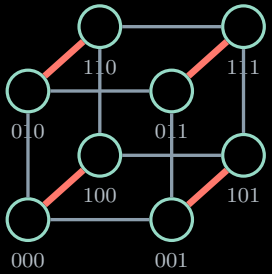
Definition

$V = \{0,1\}^d$. Join x and y iff their Hamming distance is one.

Benefit: Degree and diameter both equal $d = \log_2 N$.

Drawback: Port count grows with $\log N$.

Hypercube: add one dimension at a time



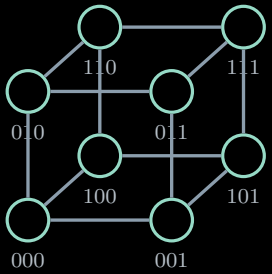
Definition

$V = \{0,1\}^d$. Join x and y iff their Hamming distance is one.

Benefit: Degree and diameter both equal $d = \log_2 N$.

Drawback: Port count grows with $\log N$.

Hypercube routing fixes the differing bits



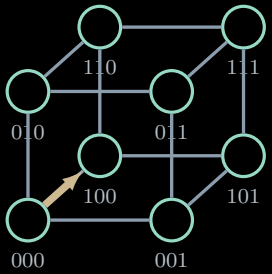
$000 \rightarrow 100 \rightarrow 110 \rightarrow 111$

Each hop fixes one destination bit.

Benefit: Simple routing, many alternate shortest paths.

Useful for: Structured peer overlays and parallel machines.

Hypercube routing fixes the differing bits



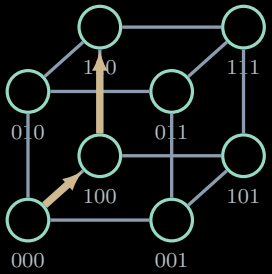
$000 \rightarrow 100 \rightarrow 110 \rightarrow 111$

Each hop fixes one destination bit.

Benefit: Simple routing, many alternate shortest paths.

Useful for: Structured peer overlays and parallel machines.

Hypercube routing fixes the differing bits



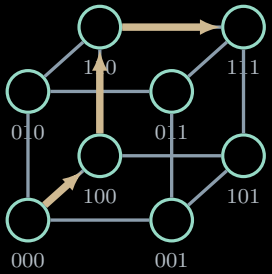
$000 \rightarrow 100 \rightarrow 110 \rightarrow 111$

Each hop fixes one destination bit.

Benefit: Simple routing, many alternate shortest paths.

Useful for: Structured peer overlays and parallel machines.

Hypercube routing fixes the differing bits



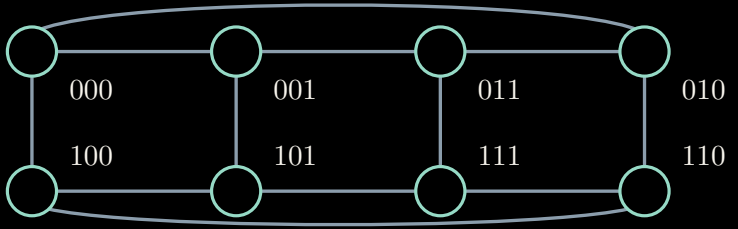
$000 \rightarrow 100 \rightarrow 110 \rightarrow 111$

Each hop fixes one destination bit.

Benefit: Simple routing, many alternate shortest paths.

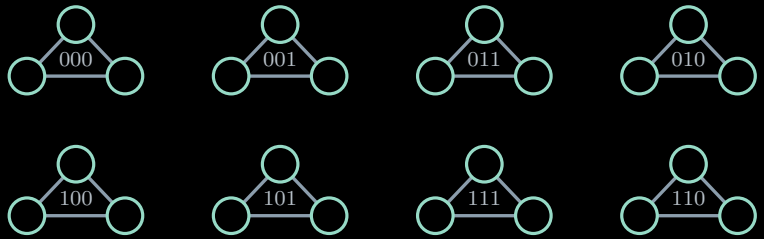
Useful for: Structured peer overlays and parallel machines.

CCC_3: replace every cube vertex



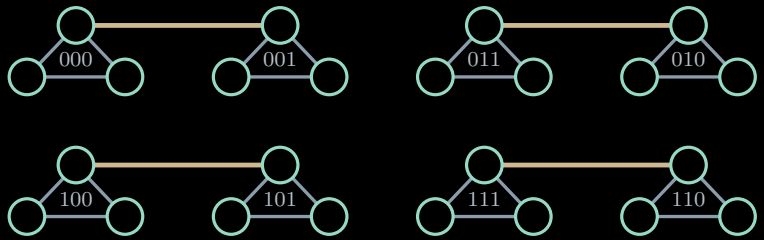
Q_3: two 4-cycles, joined by four links that flip bit 2.

CCC_3: replace every cube vertex



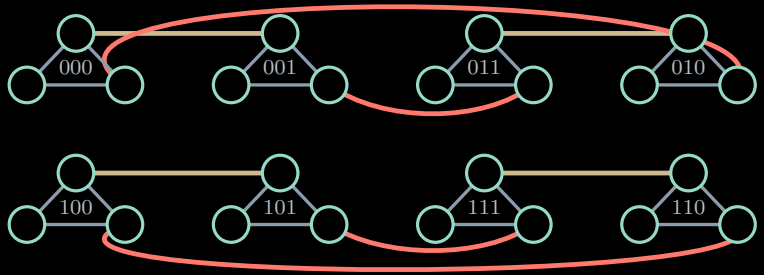
Each cube vertex becomes a three-port triangle for bits 0, 1, and 2.

CCC_3: reconnect one bit at a time



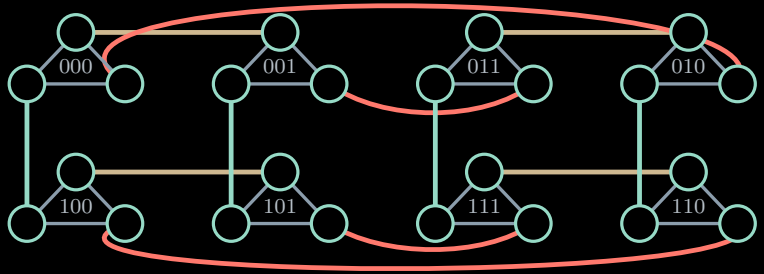
Bit 0: join port-0 pairs whose rightmost bit differs.

CCC_3: reconnect one bit at a time



Bit 1: add the four port-1 links.

CCC_3: reconnect one bit at a time



Bit 2: add four final links. CCC_3 has 24 vertices, each of degree 3.

CCC: exact vertex and edge rules

For $d \geq 3$, number the bit positions $0, \ldots, d-1$ from right to left.

Vertices

$V = \{0,1\}^d \times \mathbb{Z}_d$, so $|V| = d\,2^d$.

Cycle edges

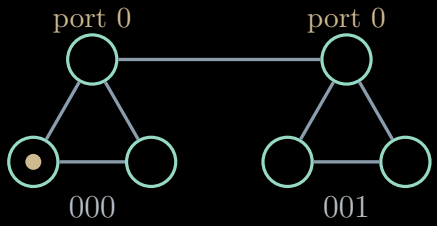
$(x, i) \sim (x, i+1 \bmod d)$.

Cube edges

$(x, i) \sim (y, i)$ when x and y differ only in bit i.

Each vertex has two neighbors in its local cycle and one cube neighbor.

CCC routing: zoom into one cube edge



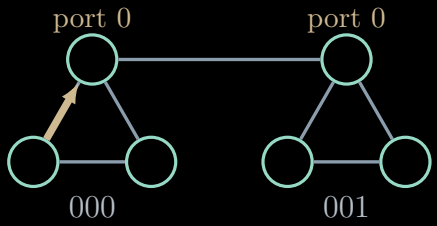
Zoom into the 000–001 edge of CCC_3. Start at $(000, 2)$.

Benefit: Three links per vertex for $d \geq 3$.

Drawback: $d\,2^d$ vertices and extra cycle hops.

Useful for: Networks with a tight port budget.

CCC routing: zoom into one cube edge



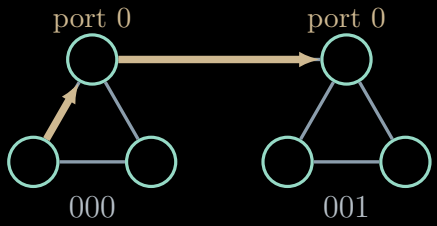
Rotate within the 000 cycle to port 0.

Benefit: Three links per vertex for $d \geq 3$.

Drawback: $d\,2^d$ vertices and extra cycle hops.

Useful for: Networks with a tight port budget.

CCC routing: zoom into one cube edge



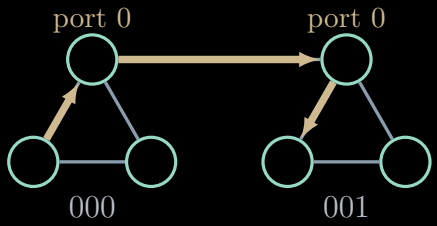
Cross the cube edge: flip bit 0 to reach $(001, 0)$.

Benefit: Three links per vertex for $d \geq 3$.

Drawback: $d\,2^d$ vertices and extra cycle hops.

Useful for: Networks with a tight port budget.

CCC routing: zoom into one cube edge



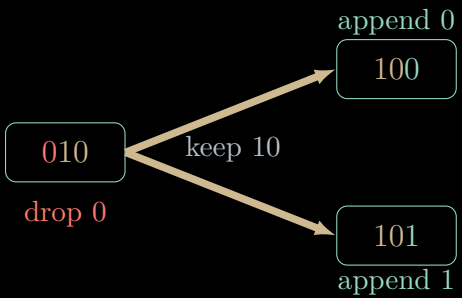
Rotate within the 001 cycle to finish at $(001, 2)$.

Benefit: Three links per vertex for $d \geq 3$.

Drawback: $d\,2^d$ vertices and extra cycle hops.

Useful for: Networks with a tight port budget.

De Bruijn: shift, then choose one new bit



Repeat this at every word.

Each d-bit word has two directed exits: drop its first bit, then append 0 or 1.

$$x = b_1 \cdots b_d$$
$$x \to b_2 \cdots b_d 0$$
$$x \to b_2 \cdots b_d 1$$

$B(2, d)$ has 2^d vertices and two outgoing arcs per vertex.

De Bruijn: reach any word in d shifts

To route from 001 to 110, append the destination bits in order.



The target bits are $1, 1, 0$. One shift admits one new bit.

De Bruijn: reach any word in d shifts

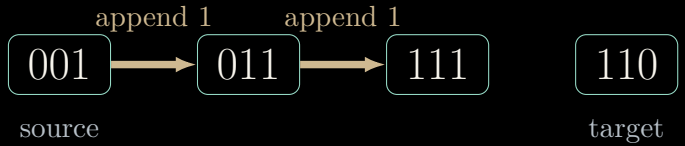
To route from 001 to 110, append the destination bits in order.



After the first shift, the suffix 1 matches the target.

De Bruijn: reach any word in d shifts

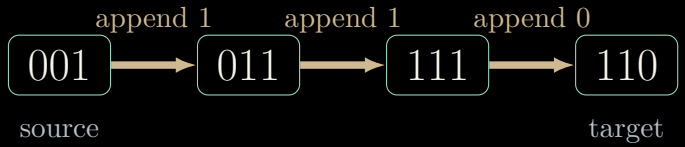
To route from 001 to 110, append the destination bits in order.



After the second shift, the suffix 11 matches.

De Bruijn: reach any word in d shifts

To route from 001 to 110, append the destination bits in order.



After d shifts, every source bit is gone.

Directed diameter is at most $d = \log_2 N$, using only two outgoing choices.

De Bruijn: the port-count tradeoff

$N = 2^d$ words

Two directed exits per word.

A destination can be reached in at most d shifts.

Useful for: Identifier-based routing and structured overlays.

Physical links need care

Incoming and outgoing arcs are different.

An undirected implementation can need up to four neighbors per word.

Drawback: Regular labels do not guarantee balanced traffic.

The formal directed graph includes self-loops at 000 and 111 for $d = 3$.

Constant degree means growing diameter



With maximum degree $\Delta > 2$, breadth-first search reaches at most

$$1 + \Delta \sum_{i=0}^{D-1} (\Delta - 1)^i$$

vertices in D hops. Therefore, a connected N-node graph needs

$$D = \Omega\left(\frac{\log N}{\log \Delta}\right).$$

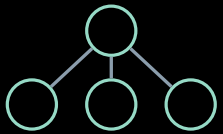
Constant degree means growing diameter

With maximum degree $\Delta > 2$,
breadth-first search reaches at most

$$1 + \Delta \sum_{i=0}^{D-1} (\Delta - 1)^i$$

vertices in D hops. Therefore, a connected N-node graph needs

$$D = \Omega\left(\frac{\log N}{\log \Delta}\right).$$



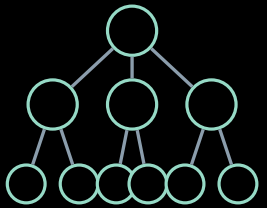
Constant degree means growing diameter

With maximum degree $\Delta > 2$,
breadth-first search reaches at most

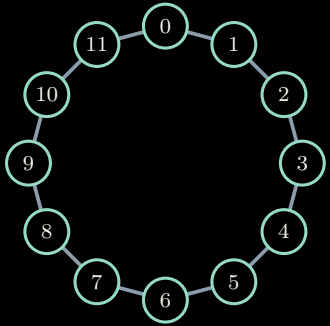
$$1 + \Delta \sum_{i=0}^{D-1} (\Delta - 1)^i$$

vertices in D hops. Therefore, a connected N-node graph needs

$$D = \Omega\left(\frac{\log N}{\log \Delta}\right).$$



One random link per node changes the ring

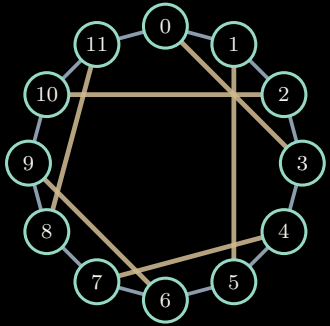


Start with C_{12}

Two ring neighbors per node.

Farthest pairs are 6 hops apart.

One random link per node changes the ring



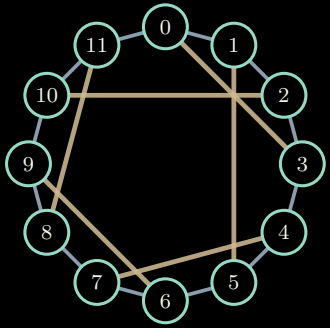
Start with C_{12}

Two ring neighbors per node.

Add a perfect matching.

Every node gains exactly one link.

One random link per node changes the ring



Start with C_{12}

Two ring neighbors per node.

Add a perfect matching.

Every node gains exactly one link.

$$D : 6 \longrightarrow 3$$

Two hops reach at most $1 + 3 + 6 = 10 < 12$ nodes.

Random regular means equal ports, random wiring

The graph model

$G_{N,d}$ is chosen uniformly from all simple graphs on N labeled vertices with degree d.

It requires Nd even and $0 \leq d < N$. Every sample has $Nd/2$ links.

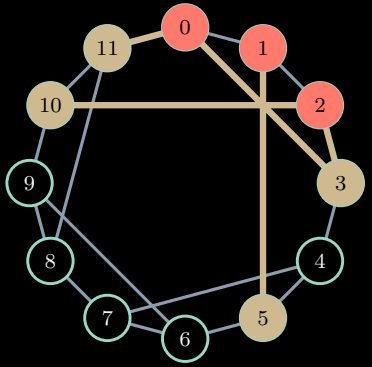
A way to sample it

Give each vertex d loose ports. Pair all ports at random. Keep the result if it has no loops or repeated links.



Same port count. Different neighbors.

An expander has no narrow region



$U = \{0, 1, 2\}$

$\Gamma(U) = \{3, 5, 10, 11\}$

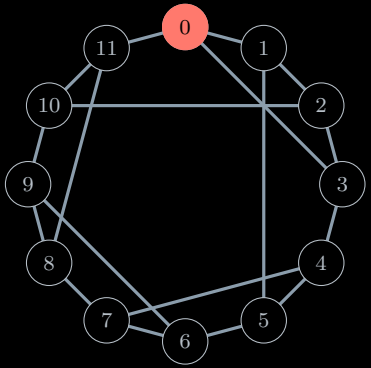
Here $|\Gamma(U)|/|U| = 4/3$.

An expander family has a constant $\epsilon > 0$ such that

$$|\Gamma(U)| \geq \epsilon |U|$$

for every U with $1 \leq |U| \leq N/2$, independent of N.

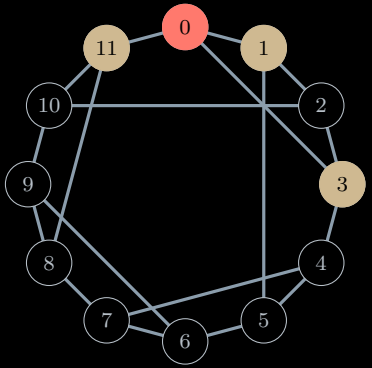
Expansion turns a few hops into broad reach



Nodes reached from 0

0 hops: 1

Expansion turns a few hops into broad reach

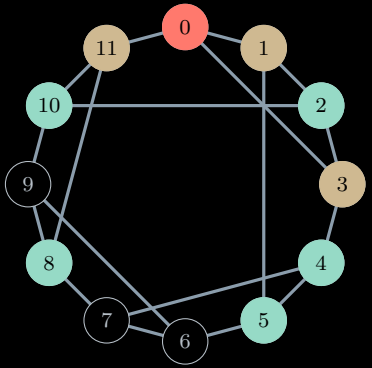


Nodes reached from 0

0 hops: 1

1 hop: 4 total

Expansion turns a few hops into broad reach



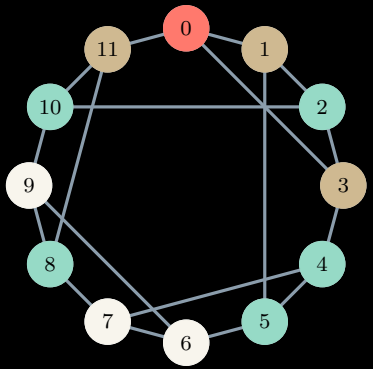
Nodes reached from 0

0 hops: 1

1 hop: 4 total

2 hops: 9 total

Expansion turns a few hops into broad reach



Nodes reached from 0

0 hops: 1

1 hop: 4 total

2 hops: 9 total

3 hops: all 12

Expansion keeps adding new neighbors.

Constant expansion implies $D = O(\log N)$.

Random regular graphs approach the hop limit

Any graph of degree d

A breadth-first tree grows by at most $d-1$ new branches after the first hop.

$$D \geq \log_{d-1} N - O(1).$$

A uniform random d-regular graph

For fixed $d \geq 3$ and growing N, with high probability,

$$D = (1 + o(1)) \log_{d-1} N.$$

Constant ports. Nearly minimum hops.

What random regular networks buy us

STyGIANet

Graph benefits

- Equal port count at every node
- Logarithmic worst-case hop count, typically
- Large boundaries around every small region, typically
- Many choices for routes across the network

Physical tradeoffs

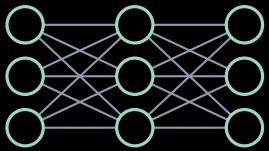
- Long, irregular cables may be costly
- Routing is less compact than on labeled graphs
- Short paths alone do not prevent hot links

These properties hold with high probability, not for every regular graph.

Which topology fits the traffic?

Traffic pattern	Starting point	Reason
Neighbor exchange	mesh or torus	regular local paths
Broadcast	tree	natural hierarchy
Peer pairs	hypercube, De Bruijn	identifier-based routing
Small fixed group	complete graph	direct links may be affordable
Parallel flows	multi-path fabric	cut capacity and path choice
Irregular peer pairs	random regular	short paths, broad cuts

Question: how should GPUs share training?



one neural network

Eight GPUs will train this model.

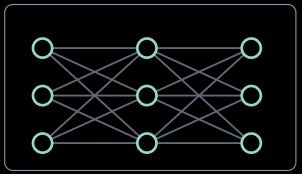
Could they split the data, layers, or neurons?

What data must cross between GPUs in each case?

Data parallelism: copy the whole network

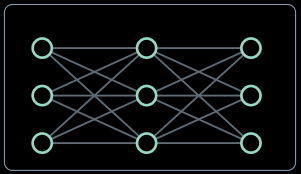
Each GPU trains a full network on a different minibatch.

batch B_0



GPU 0: complete network

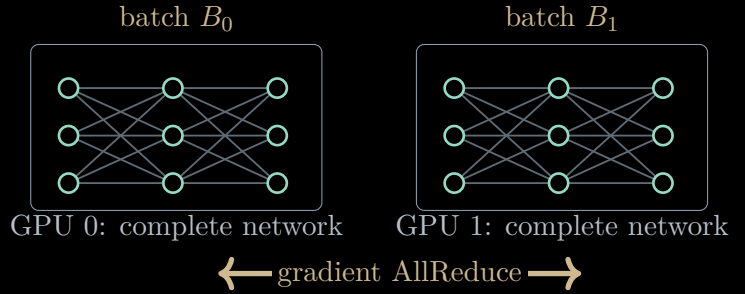
batch B_1



GPU 1: complete network

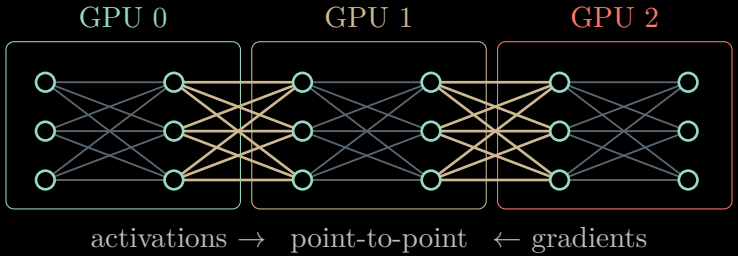
Data parallelism: copy the whole network

Each GPU trains a full network on a different minibatch.



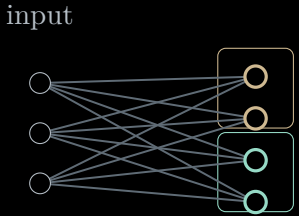
Pipeline parallelism: split the layers

Each GPU owns consecutive layers of one network.



Tensor parallelism: split one layer

This example partitions the output neurons of the middle layer.



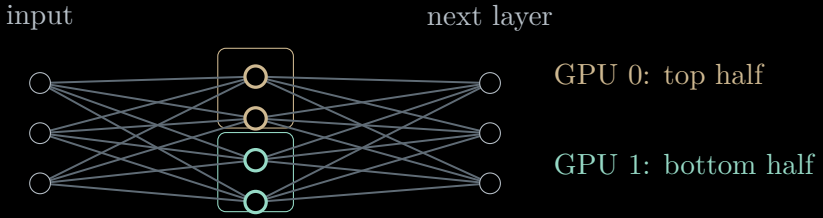
GPU 0: top half

GPU 1: bottom half

Both GPUs read the input.

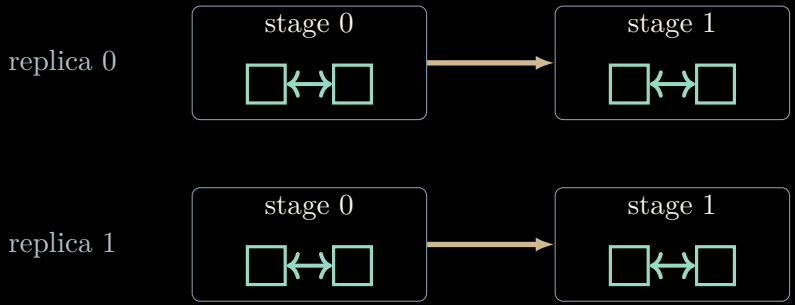
Tensor parallelism: split one layer

This example partitions the output neurons of the middle layer.



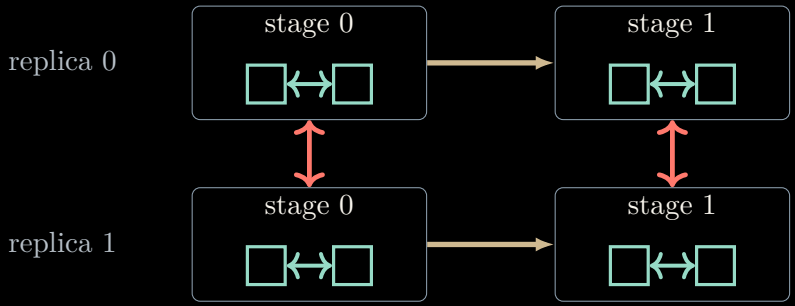
AllGather joins the activation shards for the next layer.

One training job can combine all three



Tensor: collectives Pipeline: point-to-point

One training job can combine all three



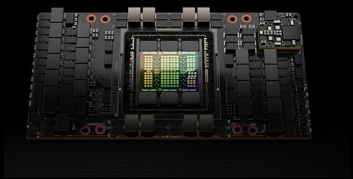
Tensor: collectives Pipeline: point-to-point Data: gradient AllReduce

Interconnects exist at several scales



Chip-level links favor high bandwidth and low latency. Node-level designs must also scale ports, cables, switch capacity, and failure domains.

The hardware behind a chip fabric



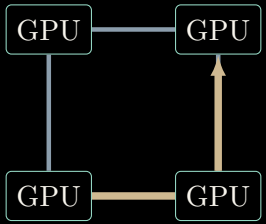
GPU module
NVIDIA H100 SXM.



Switch silicon
NVSwitch die.

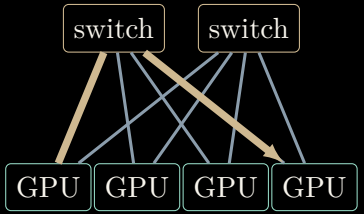
Direct links versus a switch layer

Direct neighbor links



Local wiring is simple; distant pairs need several hops.

Switched fabric



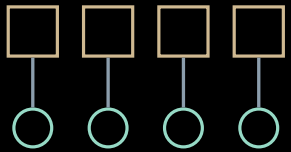
More path choice; switches, ports, and cables add cost.

Node-to-node fabrics carry collective traffic

- GPUs exchange data across servers for training.
- RDMA moves data between hosts with less CPU involvement.
- A backend fabric carries training traffic; a frontend network can serve requests.



Leaf-spine: connect every leaf to every spine



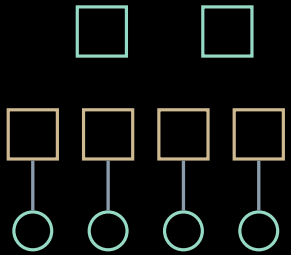
Hosts attach to leaves; every leaf reaches every spine.

Benefit:
hops and multiple paths.

Drawback: Scaling needs more spine ports.

Useful for:
Broad datacenter traffic.

Leaf-spine: connect every leaf to every spine



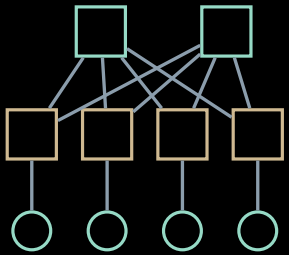
Hosts attach to leaves; every leaf reaches every spine.

Benefit:
hops and multiple paths.

Drawback: Scaling needs more spine ports.

Useful for:
Broad datacenter traffic.

Leaf-spine: connect every leaf to every spine



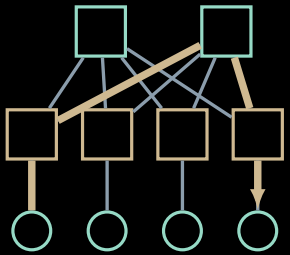
Hosts attach to leaves; every leaf reaches every spine.

Benefit:
hops and multiple paths.

Drawback: Scaling needs more spine ports.

Useful for:
Broad datacenter traffic.

Leaf-spine: connect every leaf to every spine



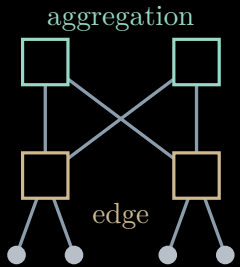
Hosts attach to leaves; every leaf reaches every spine.

Benefit:
hops and multiple paths.

Drawback: Scaling needs more spine ports.

Useful for:
Broad datacenter traffic.

K-ary FatTree: build each pod

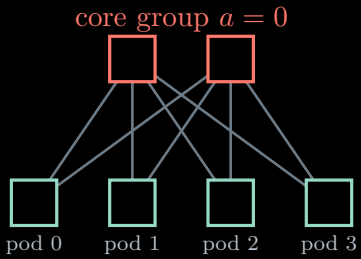


one pod when $k = 4$

1. Choose an even port count k.
2. Build k identical pods: $k/2$ edge and $k/2$ aggregation switches each.
3. From each edge switch, wire $k/2$ hosts and all $k/2$ aggregators.

This uses every edge-switch port.

K-ary FatTree: connect the core



$k = 4$: repeat for core group $a = 1$

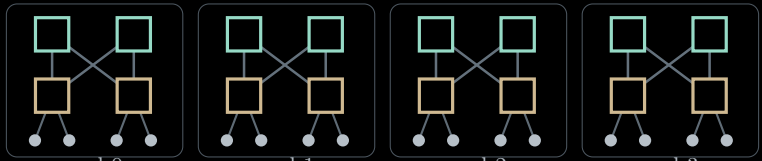
There are $k/2$ core groups. Group a contains $k/2$ switches $C_{a,j}$.

In every pod p, aggregation switch $A_{p,a}$ connects to all $C_{a,j}$.

One core port per pod.

$k^3/4$ **hosts** $5k^2/4$ switches

The complete four-ary FatTree

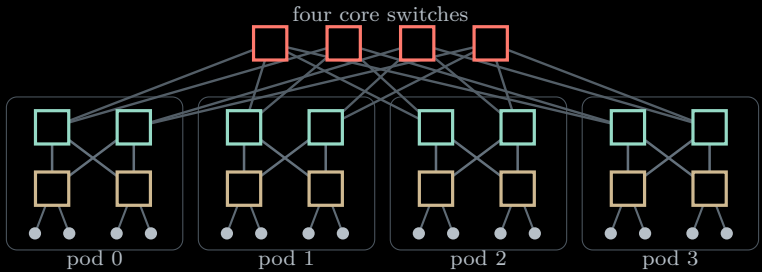


$k = 4$: 4 pods; 2 edge and 2 aggregation switches per pod; 4 core switches; 16 hosts.

Each edge switch connects its two hosts and both aggregation switches in its pod.

Benefit: Four paths. **Drawback:** More cables. **Useful for:** Large-scale datacenters.

The complete four-ary FatTree

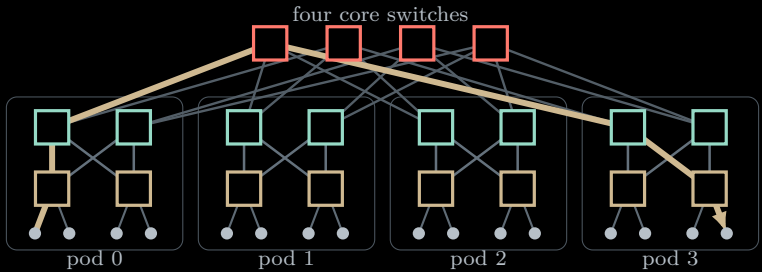


$k = 4$: 4 pods; 2 edge and 2 aggregation switches per pod; 4 core switches; 16 hosts.

Aggregation switch a connects to both switches in core group a.

Benefit: Four paths. **Drawback:** More cables. **Useful for:** Large-scale datacenters.

The complete four-ary FatTree



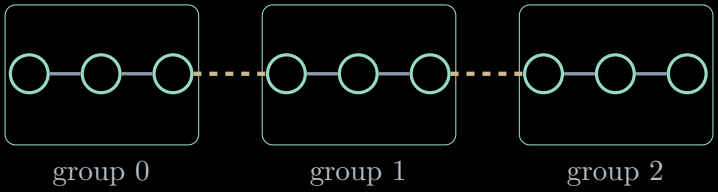
$k = 4$: 4 pods; 2 edge and 2 aggregation switches per pod; 4 core switches; 16 hosts.

Different-pod hosts have four equal-cost paths.

Benefit: Four paths. **Drawback:** More cables. **Useful for:** Large-scale datacenters.

Question: what if most traffic stays local?

Suppose each group talks mostly within itself.



Should ports favor local links or global links? Which traffic pays?

Two ways to connect local groups

Dragonfly

Unit: a group of routers.

Inside: every router reaches every other router.

Between: one global link per group pair in the full-size construction.

HammingMesh

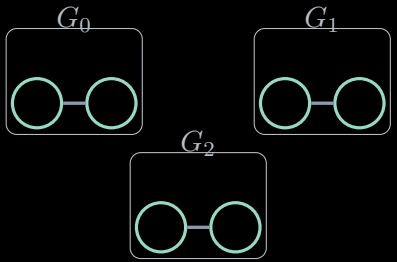
Unit: a board of accelerators.

Inside: short links form a 2D mesh.

Between: switched networks connect boards along rows and columns.

Dragonfly favors low global hop count; HammingMesh favors high local bandwidth.

Dragonfly: strong groups, fewer global links



Toy example: $a = 2$, $h = 1$, $g = 3$.

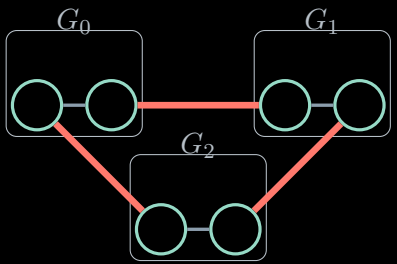
Each router has one local port and one global port.

A minimal route uses local $\rightarrow$ global $\rightarrow$ local.

Benefit: One global hop per minimal intergroup route.

Drawback: Global links can become hot spots.

Dragonfly: strong groups, fewer global links



Toy example: $a = 2$, $h = 1$, $g = 3$.

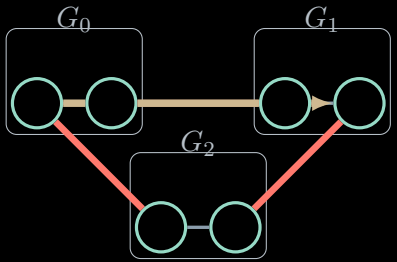
Each router has one local port and one global port.

A minimal route uses local $\rightarrow$ global $\rightarrow$ local.

Benefit: One global hop per minimal intergroup route.

Drawback: Global links can become hot spots.

Dragonfly: strong groups, fewer global links



Toy example: $a = 2$, $h = 1$, $g = 3$.

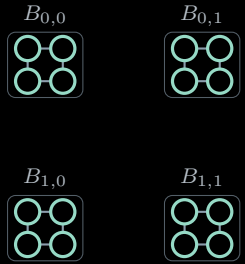
Each router has one local port and one global port.

A minimal route uses local $\rightarrow$ global $\rightarrow$ local.

Benefit: One global hop per minimal intergroup route.

Drawback: Global links can become hot spots.

HammingMesh: row and column fabrics



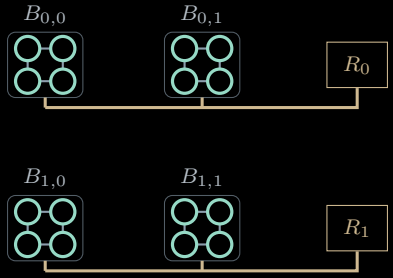
1. Build a fast 2D mesh on each board.

Across both axes, a route can use an intermediate board.

Benefit: High local bandwidth.

Drawback: Shared global switches.

HammingMesh: row and column fabrics



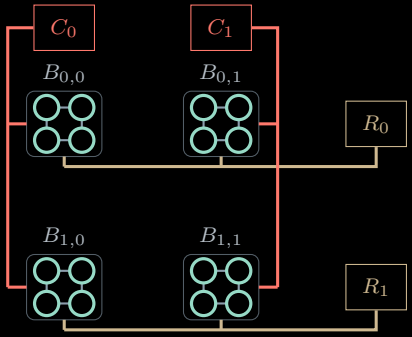
2. Add switches across each row.

Across both axes, a route can use an intermediate board.

Benefit: High local bandwidth.

Drawback: Shared global switches.

HammingMesh: row and column fabrics



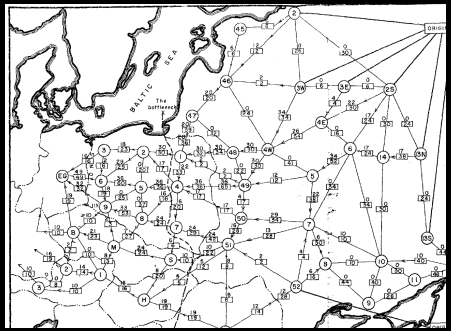
3. Add switches across each column.

Across both axes, a route can use an intermediate board.

Benefit: High local bandwidth.

Drawback: Shared global switches.

Where could limited damage disrupt rail traffic?



1955: rail interdiction

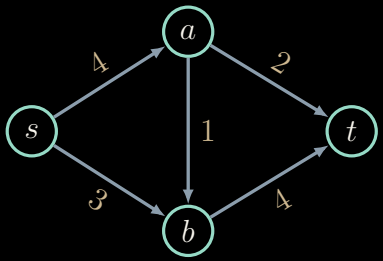
Air Force planners wanted to know which attacks would most reduce Soviet rail capacity.

Which links are bottlenecks?

Estimate the flow that survives each damage scenario.

Historical map: Harris and Ross, RAND RM-1573 (1955).

Replace the map with a capacity graph



$G = (V, E)$: routers or stations and directed links.

$c_e \geq 0$: capacity of link e per unit time.

s: source. t: destination.

Numbers are a classroom example, not values transcribed from the map.

How much can travel from s to t at once?

A feasible flow obeys two rules

Capacity

For every directed link e,

$$0 \le f_e \le c_e.$$

A link cannot carry more than its rate limit.

Conservation

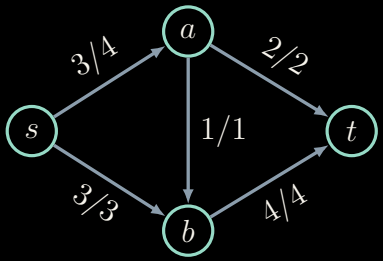
At every node except s and t,

$$\sum_{e \in \text{in}(v)} f_e = \sum_{e \in \text{out}(v)} f_e.$$

Traffic cannot disappear or be created in transit.

The flow value is the net rate leaving s.

Three routes carry six units



Path decomposition

$s \to a \to t$: 2 units

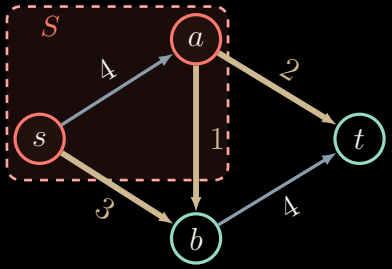
$s \to a \to b \to t$: 1 unit

$s \to b \to t$: 3 units

$|f| = 2 + 1 + 3 = 6$

Each link label is flow/capacity.

A cut limits every feasible flow



Set $S = \{s, a\}$ contains the source, but not the destination.

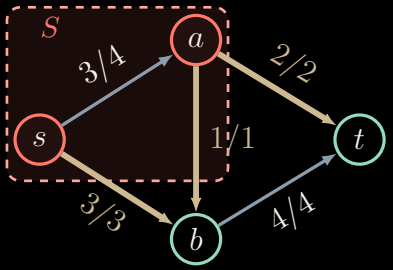
Only links leaving S count:

$$c(\delta^+(S)) = 3 + 1 + 2 = 6.$$

Removing these links disconnects s from t.

$|f| \leq c(\delta^+(S))$.

Maximum flow equals minimum cut



Flow $= 6$

Three paths carry $2 + 1 + 3$.

Cut capacity $= 6$

Gold links carry $3 + 1 + 2$ and are saturated.

The matching values prove optimality.

$$\max_f |f| = \min_{s \in S,\, t \notin S} c(\delta^+(S)) = 6.$$

Why the flow and cut values meet

STyGIANet

1. Start with zero flow. Send more along any s-t path with spare capacity.
2. A backward residual link can undo earlier flow and reroute it.
3. When no residual s-t path remains, let S be the nodes still reachable from s.

Every link from S to $V \setminus S$ is saturated.
The flow value equals the capacity of this cut.

Traffic matrix

source \ destination	A	B	C	D
A	–	0	0	2
B	0	–	0	0
C	0	1	–	0
D	0	0	0	–

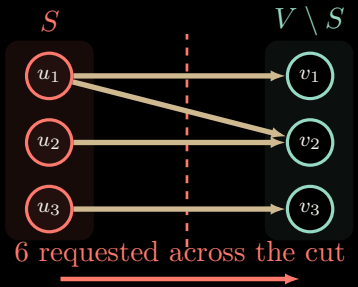
Each cell M_{uv} is the rate requested from source u to destination v.

$M_{AD} = 2$: A sends 2 to D.

$M_{CB} = 1$: C sends 1 to B.

Each nonzero cell is a commodity. Their chosen routes share link capacity.

Sparsest cut measures demand pressure



Cut capacity: $c(S, \bar{S}) = 4$.

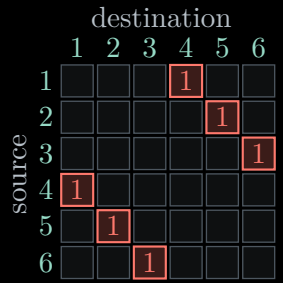
Demand across the cut: $M(S, \bar{S}) = 6$.

$$\delta(S) = \frac{4}{6} = \frac{2}{3}.$$

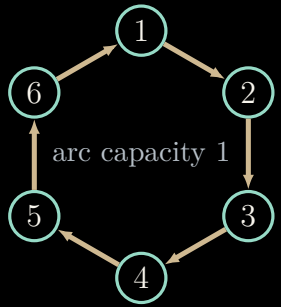
The sparsest cut value is the smallest such ratio.

$$\delta = \min_{S:\, M(S,\bar{S})>0} \delta(S), \text{ where } \delta(S) = c(S, \bar{S})/M(S, \bar{S}).$$

Concurrent flow scales all demands



θM



Scale all six demands together. What is the largest feasible θ?

Maximum concurrent flow scales every pair

$x_p \geq 0$ is the rate on path p; $\theta \geq 0$ scales demand; $\mathcal{P}_{uv}$ is the set of u-v paths.

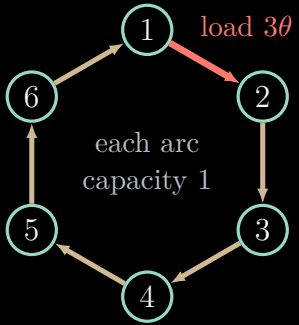
Objective $\quad \max \ \theta$

Capacity $\quad \sum_{p \ni e} x_p \leq c_e \quad$ for every link e

Demand $\quad \sum_{p \in \mathcal{P}_{uv}} x_p = \theta M_{uv} \quad$ for every pair (u, v)

The same θ multiplies every nonzero entry of the traffic matrix.

A directed ring exposes shared load



Each node sends one unit to the node opposite it:

$$1 \to 4,\ 2 \to 5, \ldots,\ 6 \to 3.$$

The highlighted arc carries three requests: $1 \to 4$, $5 \to 2$, and $6 \to 3$.

At scale θ, its load is 3θ and its capacity is 1.

So $3\theta \le 1$. Every arc has the same load, giving $\theta^* = 1/3$.

Maximum link utilization keeps demands fixed

$x_p \geq 0$ is the path rate; $\rho \geq 0$ is the largest link load divided by capacity.

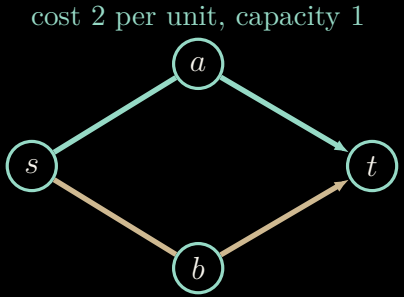
Objective $\quad \min \ \rho$

Link load $\quad \sum_{p \ni e} x_p \leq \rho c_e \quad$ for every link e

Demand $\quad \sum_{p \in \mathcal{P}_{uv}} x_p = M_{uv} \quad$ for every pair (u, v)

Ring example: $\rho^* = 3$. For nonzero demands and fractional routing, $\theta^* = 1/\rho^*$.

Min-cost flow



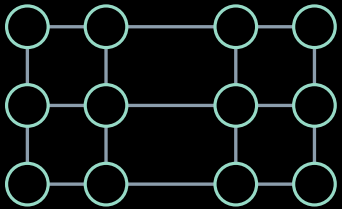
Send three units from s to t.

Use the cheap path for one unit and the other path for two.

Minimum cost $= 1(2) + 2(4) = 10$.

Link costs add along a path; capacities still apply.

Bisection bandwidth

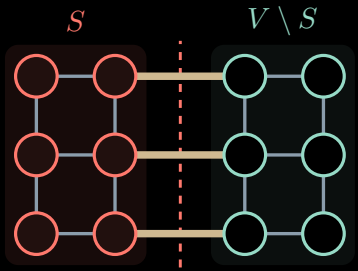


Split the nodes into two equal halves. Sum the capacities of links between them.

Bisection bandwidth is the smallest such sum.

Unlike an s-t cut, no source or destination is specified.

Bisection bandwidth



Split the nodes into two equal halves. Sum the capacities of links between them.

Bisection bandwidth is the smallest such sum.

Here it is three unit-capacity links.

Unlike an s-t cut, no source or destination is specified.

How to compare candidate fabrics

- Latency: path time under load
- Bisection: balanced cut capacity
- Max flow: one sender-receiver
- Concurrent flow: all pairs
- MLU: busiest link fraction
- Cost: ports, cables, power
- Resilience: after failures
- Routing: path balance

Compare topologies under the traffic they must carry.

Choose the bottleneck you can afford

If you need…	Consider…	Watch for…
low port count	tree, mesh, Dragonfly	narrow cuts and longer routes
very low hop count	complete, leaf-spine	ports, links, switches
regular local wiring	mesh, torus	distant traffic
collective bandwidth	FatTree, HammingMesh	switch stages and cost
peer routing	hypercube, De Bruijn	overlay maintenance

The workload and physical layout decide the final design.

What to remember

1. Degree, diameter, and expansion constrain possible paths.
2. For one pair, maximum flow equals minimum s-t cut.
3. For many pairs, cut ratios bound concurrent throughput.
4. Traffic and physical cost determine the useful design.

Next: collective algorithms compete for capacity on these paths.

References and further reading

- J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.
- S. Saroiu, P. K. Gummadi, and S. D. Gribble, “A Measurement Study of Peer-to-Peer File Sharing Systems,” *Proc. SPIE*, vol. 4673, pp. 156–170, 2002. doi: 10.1117/12.449977.
- T. Yamada, T. Ohtsuka, A. Watanabe, and S. Ueno, “On Sequential Diagnosis of Multiprocessor Systems,” *Discrete Applied Mathematics*, vol. 146, no. 3, pp. 311–342, 2005. doi: 10.1016/j.dam.2004.08.008.

References and further reading

- M. Al-Fares, A. Loukissas, and A. Vahdat, “A Scalable, Commodity Data Center Network Architecture,” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63–74, 2008. doi: 10.1145/1402946.1402967.
- J. Kim, W. J. Dally, S. Scott, and D. Abts, “Technology-Driven, Highly-Scalable Dragonfly Topology,” in *Proc. ISCA*, pp. 77–88, 2008. doi: 10.1109/ISCA.2008.19.
- T. Hoefler et al., “HammingMesh: A Network Topology for Large-Scale Deep Learning,” in *Proc. SC*, pp. 1–18, 2022. doi: 10.1109/SC41404.2022.00016.

References and further reading

- B. Bollobas and F. R. K. Chung, “The Diameter of a Cycle Plus a Random Matching,” *SIAM Journal on Discrete Mathematics*, vol. 1, no. 3, pp. 328–333, 1988. doi: 10.1137/0401033.
- B. Bollobas and W. Fernandez de la Vega, “The Diameter of Random Regular Graphs,” *Combinatorica*, vol. 2, pp. 125–134, 1982. doi: 10.1007/BF02579310.
- S. Hoory, N. Linial, and A. Wigderson, “Expander Graphs and Their Applications,” *Bulletin of the American Mathematical Society*, vol. 43, no. 4, pp. 439–561, 2006. doi: 10.1090/S0273-0979-06-01126-8.

References and further reading

- T. E. Harris and F. S. Ross, *Fundamentals of a Method for Evaluating Rail Net Capacities*, RAND Research Memorandum RM-1573, 1955.
- L. R. Ford, Jr., and D. R. Fulkerson, “Maximal Flow Through a Network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. doi: 10.4153/CJM-1956-045-5.
- D. P. Bertsekas, *Linear Network Optimization: Algorithms and Codes*. MIT Press, 2003.
- F. T. Leighton and S. Rao, “Multicommodity Max-Flow Min-Cut Theorems and Their Use in Designing Approximation Algorithms,” *Journal of the ACM*, vol. 46, no. 6, pp. 787–832, 1999. doi: 10.1145/331524.331526.